

String Builder & String Buffer

- * These two classes are work as a string but these two are mutable (values can be updated), but not create new strings.
- * These will not waste our memory.
- * These two provides `reverse()` method.
- * String Buffer - Thread Safe (Synchronized) - Slower process (due to lock).
- * String Builder - Not Thread Safe - It introduced in 5th version.
 - Not Synchronized.
 - Faster.

Ex: package com;

public class Reverse{

public static void main(String[] args){

String s = "Java";

String rev = "";

for (int i = s.length-1; i >= 0; i--){

rev += s.charAt(i);

output: aVaJ

}

System.out.println(rev);

}

}

Syntax:

* String Buffer `sb = new StringBuffer("Java");`

`sb.reverse();` output: aVaJ

* String Builder `sb = new StringBuilder("Java");`

Methods:

* Default capacity of String Builder & String Buffer is 16.

String Buffer `sb = new StringBuffer("Java");`

initial capacity = 4 + 16 = 20

* If we take 22 characters, (adding two characters to sb);

capacity = (old capacity + new size) * 2

(20 + 2) * 2 = 44;

package com;

public class Buffer {

public static void main (String[] args) {

StringBuffer sb = new StringBuffer("Java ~~World~~");

* sb.append("World");

output: Java World.

syso (sb);

Java, World.

* sb.insert("4", " ");

Java, Programming

syso (sb);

Java, Programming.

// Changes the entire word.

16 (length)

* sb.replace(6, 11, "Programming");

32 (capacity)

(starting index) (ending index +1)

Java, Programming

syso (sb);

1 2 3 4 5 6 7 8 9.

25 (length)

* To delete some things.

50 (capacity)

sb.delete(5, 6);

syso (sb);

* syso (sb.length());

* syso (sb.capacity());

* sb.append("123456789");

* syso (sb);

* syso (sb.length());

* syso (sb.capacity());

String Builder sb2 = new String Builder (50);

syso (sb2.capacity()); → output: 50

* String s = sb.toString();

// String Buffer converted to String.

}

* ³ Here toString() method is overridden so we are getting direct values instead of getting hashCode even if we print objects (sb).

* Math - class in Java. -

⇒ Methods in Math class:

Ex: package com;

public class Test {

public static void main (String[] args) {

// abs() - absolute method - It takes absolute value.
- It takes + & -ve values as +ve

System.out.println(Math.abs(-10));

Output: 10.

// Math.max() - prints maximum value in given 2 values.

System.out.println(Math.max(10, 11));

// Math.min() - prints minimum value in given 2 values.

System.out.println(Math.min(10, 11));

// Math.round() - it will round to nearest value (integer).

System.out.println(Math.round(5.4));

// Math.ceil() - it will give next possible integer.

System.out.println(Math.ceil(5.1));

// Math.floor() - it gives previous possible integer.

System.out.println(Math.floor(5.9));

// Math.pow() - Number to the power.

System.out.println(Math.pow(2, 3));

// Math.sqrt() - square root.

System.out.println(Math.sqrt(25));

// Math.cbrt() - cube root.

System.out.println(Math.cbrt(125));

* Trigonometric values also we can do this class.

* System.out.println(Math.tan(Math.PI/4));

Output:
0.99999999...

// Math.random() - this will return a random value from 0.0 to 1.0.

System.out.println(Math.random());

Output: Random output.
(if we run multiple times).

* We can give range to this random values.

// Math.random() * 100;

sysout(Math.random() * 100);

* **Random** - class in Java. $\Rightarrow$ we can give range of random values.

Random random = new Random();

sysout(random.nextInt(100)); output: Random outputs b/w 0 & 100

sysout(random.nextInt(10, 15)); output: Random outputs b/w 10 & 15

* **java.lang** is called as default package.

$\rightarrow$ all classes stored in this package.

$\rightarrow$ String, Math, Exception, Object.

$\rightarrow$ We need not to import this package (due to it will come from default package).

* main method.

public static void main (String [] args) {
 ↓ ↓ ↓ ↓ ↓
(access modifiers) (keyword) (return type) (method name) (Array to store multiple values) (variable name)
 ↓ ↓ ↓
 (static) (Instance method)
 System.out.println ();
 ↓ ↓ ↓
 (class) (variable) (print stream method)

public static final PrintStream out = null

* Out is a type of PrintStream variable.

* **System** - class (java.lang).

Out - Variable, PrintStream obj.
 ↓
 (static)

println - printStream method, Instance method.

* **Var Args** (Variable arguments)

* This can be passed inside a method.

public static void main (String... args) {

 ↓
 (It works as an Array)

}

* It can accept multiple number of arguments.